

PREFACE FOR FACULTY

Filter realization transforms a mathematical transfer function $H(z)$ into a physical hardware/software signal flow graph composed of adders, multipliers, and unit delay registers z^{-1} . For Finite Impulse Response (FIR) filters, realization structures directly dictate execution speed, VLSI silicon area, pipelining capability, and resilience to finite word-length quantization.

Pedagogical Strategy:

1. Introduce standard Signal Flow Graph (SFG) conventions: Branches, summing nodes, delay nodes z^{-1} , and gain multipliers.
 2. Formulate the **Direct Form (Transversal / Tapped Delay Line)** structure: $y[n] = \sum_{k=0}^{M-1} b_k x[n - k]$.
 3. Apply **Tellegen's Theorem / Flow Graph Reversal Theorem** to derive the **Transposed Direct Form**, explaining why it reduces accumulator critical path delay in high-speed hardware.
 4. Formulate the **Cascade Realization**, factoring an M^{th} -order polynomial into 2nd-order real sections (biquads).
 5. Compare hardware resource requirements: M multipliers, $M - 1$ adders, $M - 1$ delays.
-

1. LEARNING OBJECTIVES

By the end of this lecture, students will be able to:

1. **Draw** Direct Form and Transposed Direct Form signal flow graphs for arbitrary FIR transfer functions.
 2. **Apply** the Flow Graph Reversal Theorem to transpose digital filter networks.
 3. **Factor** high-order FIR transfer functions into cascade second-order sections.
 4. **Compare** Direct and Cascade realizations in terms of multiplier count, memory delays, and coefficient sensitivity.
-

2. MATHEMATICAL FOUNDATIONS

2.1 FIR Transfer Function & Direct Form

An FIR filter of length M (order $N = M - 1$) is described by:

$$y[n] = \sum_{k=0}^{M-1} b_k x[n - k] = b_0 x[n] + b_1 x[n - 1] + \cdots + b_{M-1} x[n - M + 1]$$
$$H(z) = \sum_{k=0}^{M-1} b_k z^{-k} = b_0 + b_1 z^{-1} + b_2 z^{-2} + \cdots + b_{M-1} z^{-(M-1)}$$

- **Direct Form Structure:** Consists of a tapped delay line of $M - 1$ registers, M parallel multiplier branches feeding a multi-input summation bus.

2.2 Transposed Direct Form (Flow Graph Reversal)

By the Flow Graph Reversal Theorem (Tellegen's Theorem for linear networks):

1. Reverse the direction of all signal branches.
2. Summing nodes become branching nodes; Branching nodes become summing nodes.
3. Interchange the input $x[n]$ and output $y[n]$. The state equations for the Transposed Direct Form are:

$$\begin{aligned} y[n] &= b_0 x[n] + v_1[n-1] \\ v_k[n] &= b_k x[n] + v_{k+1}[n-1], \quad k = 1, 2, \dots, M-2 \\ v_{M-1}[n] &= b_{M-1} x[n] \end{aligned}$$

- **Engineering Advantage:** The input $x[n]$ is broadcast simultaneously to all multipliers. Adders are distributed between delay registers, eliminating the long critical-path adder tree of the standard direct form and facilitating high-speed pipelining.

2.3 Cascade Realization

Factoring $H(z)$ into a product of second-order real polynomials:

$$H(z) = b_0 \prod_{k=1}^K (1 + \beta_{1k} z^{-1} + \beta_{2k} z^{-2}), \quad K = \lfloor M/2 \rfloor$$

Each second-order section realizes a pair of complex conjugate zeros (z_k, z_k^*) : $1 + \beta_{1k} z^{-1} + \beta_{2k} z^{-2} = (1 - z_k z^{-1})(1 - z_k^* z^{-1})$ Where $\beta_{1k} = -2\text{Re}(z_k)$ and $\beta_{2k} = |z_k|^2$.

3. WORKED NUMERICAL EXAMPLES

Example 15.1: Direct and Cascade Realization of a 4th-Order FIR Filter

Problem: Realize the FIR filter given by transfer function:

$$H(z) = 1 + 2.5z^{-1} + 2.75z^{-2} + 1.25z^{-3} + 0.25z^{-4}$$

in (a) Direct Form, (b) Transposed Direct Form, and (c) Cascade Form.

Solution: (a) Direct Form: $y[n] = x[n] + 2.5x[n-1] + 2.75x[n-2] + 1.25x[n-3] + 0.25x[n-4]$. Requires 4 delay elements z^{-1} , 5 multipliers (or 4 non-trivial), and 4 two-input adders.

(b) Transposed Direct Form: $y[n] = x[n] + v_1[n-1]$ $v_1[n] = 2.5x[n] + v_2[n-1]$ $v_2[n] = 2.75x[n] + v_3[n-1]$ $v_3[n] = 1.25x[n] + v_4[n-1]$ $v_4[n] = 0.25x[n]$.

(c) Cascade Form: Factor $H(z)$ into two 2nd-order sections:

$$H(z) = (1 + 1.5z^{-1} + 0.5z^{-2})(1 + 1.0z^{-1} + 0.5z^{-2})$$

- Section 1: $H_1(z) = 1 + 1.5z^{-1} + 0.5z^{-2}$ (zeros at $z = -0.5, -1.0$)
- Section 2: $H_2(z) = 1 + 1.0z^{-1} + 0.5z^{-2}$ (zeros at $z = -0.5 \pm j0.5$) The system is realized as the direct cascade of $H_1(z)$ followed by $H_2(z)$.

4. UNIVERSITY EXAMINATION QUESTIONS & MARKING RUBRIC

Question 1 (15 Marks)

(a) State the Flow Graph Reversal Theorem and explain how it is used to derive the Transposed Direct Form FIR structure. (6 Marks) (b) Realize the linear-phase FIR filter $H(z) = 1 - \frac{1}{2}z^{-1} + \frac{3}{4}z^{-2} - \frac{1}{2}z^{-3} + z^{-4}$ in Cascade Form using real second-order sections. Draw the complete SFG. (9 Marks)

Model Answer & Step-by-Step Marking Rubric:

- **Part (a):**
 - Flow Graph Reversal Theorem rules (reversing branches, swapping nodes, swapping I/O) (3 Marks)
 - Derivation of state equations and explanation of critical path pipelining advantage (3 Marks)
- **Part (b):**
 - Factor $H(z)$ by finding roots or grouping quadratic factors:
$$H(z) = (1 - z^{-1} + z^{-2})(1 + 0.5z^{-1} + z^{-2}) = 1 - 0.5z^{-1} + 0.75z^{-2} - 0.5z^{-3} + z^{-4}$$
 (4 Marks)
 - Section 1: $H_1(z) = 1 - z^{-1} + z^{-2}$
 - Section 2: $H_2(z) = 1 + 0.5z^{-1} + z^{-2}$ (2 Marks)
 - Neatly drawn SFG showing cascade interconnection with labeled branch gains and unit delays (3 Marks)

5. PYTHON VERIFICATION SCRIPT

```
import numpy as np
import scipy.signal as signal

b = [1.0, 2.5, 2.75, 1.25, 0.25]
sos = signal.tf2sos(b, [1.0])
print("Cascade SOS Sections:")
print(sos)
```